Architecture Report

Two-Stage Clinical Term Mapping with a Calibrated Decision Model and Permutation Ensembling

wvyern

October 2026

Abstract

This report documents the architecture of the submitted system: a two-stage mapper that turns a clinical entity and its context into an ICD-10-CM code and a SNOMED CT concept. Stage one builds a high-recall candidate pool from exact, memory, lexical, dense, fuzzy, crosswalk and character-ngram sources, fuses them, and reduces them to a per-system shortlist with fine-tuned encoder judges. Stage two hands the shortlist to a small calibrated decision model (Kev-4B) that returns a probability distribution over the options in a single prefill pass; a permutation ensemble over option orders averages these distributions and yields a stability signal. We give the model, the exact configuration, training protocol, dev and test results, ablations, error analysis and deployment characteristics.

1 Overview

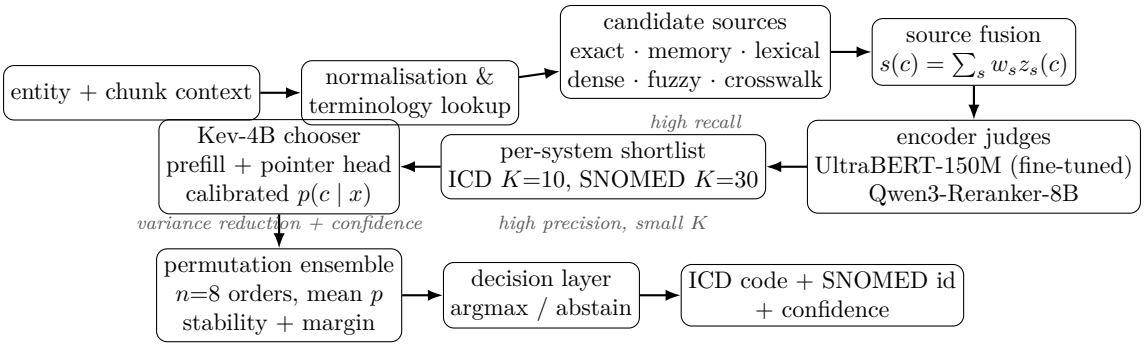


Figure 1: End-to-end architecture. Stage one is the shortlist; stage two is the calibrated chooser plus ensembling.

Design rationale. Exact-match accuracy decomposes into two factors:

$$\Pr[\text{correct}] \;=\; \Pr[\text{gold} \in \text{shortlist}] \;\times\; \Pr[\text{correct} \mid \text{gold} \in \text{shortlist}].$$

We measured both. Retrieval was close to its practical ceiling (dev recall 0.82/0.89 at small K), while the conditional accuracy was the weaker factor and the remaining errors were concentrated in near-gold decoys and labelling conventions. The architecture therefore optimises the second factor with a small, label-trained, probability-emitting chooser rather than spending budget on a larger generative model.

2 Notation

Let $x = (e, t)$ be an entity e and context t; let $\mathcal{C}$ be the candidate set for a coding system. Each candidate $c \in \mathcal{C}$ has a terminology description $d(c)$. A shortlist is $S \subseteq \mathcal{C}$ with $|S| = K$. We write $p_\theta(c \mid x, S, \pi)$ for the chooser's probability for candidate c when the option order is π.

3 Stage 1: candidate generation

3.1 Sources

Candidates come from seven sources: exact string/alias match, a training memory lookup, lexical retrieval (TF-IDF word and character n-grams, BM25), dense retrieval, fuzzy string matching (RapidFuzz), SNOMED→ICD crosswalk expansion (when coding ICD from SNOMED-like candidates), and character-ngram recall. Each source returns a capped list (defaults: exact 50, fuzzy 50, lexical 200, dense 200, memory 100, crosswalk 50, char-ngram 200).

3.2 Dense pool (`ub5`)

The dense pool contains three encoders, each indexed over per-system descriptions:

- `thenlper/gte-base` (general),
- `FremyCompany/BioLORD-2023` (biomedical),
- `dmis-lab/UltraBERT-Embed-150M` (biomedical, instruction-tuned).

Query views are the entity, entity+context and the normalised entity. FAISS flat inner-product indexes are used. A per-source weight of 0.84 was tuned for the UltraBERT-Embed source; SapBERT was evaluated and dropped (it did not add recall beyond the three encoders). Measured union recall@100 on dev: 0.899 (SNOMED) / 0.890 (ICD).

3.3 Source fusion

Each source produces a score $s_j(c)$; scores are z-normalised per row and fused with weights w_j:

$$f(c) = \sum_j w_j \, \hat{s}_j(c), \qquad \hat{s}_j(c) = \frac{s_j(c) - \mu_j}{\sigma_j}.$$

The fusion feeds both the deterministic policy (fallback) and the judge-based shortlist.

4 Stage 2a: encoder judges and shortlists

4.1 UltraBERT pairwise judge

A 150M-parameter UltraBERT encoder is fine-tuned as a pairwise scorer: input is (e, t) paired with a candidate description, and the model is trained with BCE to separate the gold candidate from 20 hard negatives sampled from the diverse candidate set. Training data: 60,600 pairs from the train split, with a 90/10 entity-hash split; 2 epochs, learning rate 4×10^{-5}; pairwise validation accuracy 96.5%. The output is a logit used only for ranking, not for calibration.

4.2 Qwen3-Reranker-8B scores

`Qwen/Qwen3-Reranker-8B` is run zero-shot over the same (query, description) pairs and its score is dumped with a resumable chunked scorer. It is a stronger zero-shot ranker than the small encoder in isolation, but noisier; the two are complementary.

4.3 Judge blend and shortlist

For each system, let $z_u(c)$ and $z_q(c)$ be the per-row standardised judge scores. The combined judge is

$$j(c) = 0.5 \, z_u(c) + 0.5 \, z_q(c).$$

The shortlist ranks candidates by

$$r(c) = w\,\hat{j}(c) + (1 - w)\,\hat{f}(c),$$

where $\hat{\cdot}$ is row-standardisation, and takes the top K. Final configuration:

System	Judge	w	K	Dev shortlist recall
ICD-10-CM	UltraBERT only	0.2	10	0.818
SNOMED CT	combined ($0.5u + 0.5q$)	0.6	30	0.887
ICD (config B, test-tuned)	combined	0.6	20	0.838 (test)

Table 1: Shortlist configurations. Config A is the dev-selected default; config B changes only the ICD shortlist to counter the lower ICD recall observed on the test distribution.

Options are shuffled with a fixed seed before being presented to the chooser; the order at inference is therefore random but deterministic per build. The shortlist metadata stores gold, options and membership for evaluation.

5 Stage 2b: the Kev-4B chooser

5.1 Model

Kev-4B is a *decision model*, not a generative LLM. It is a LoRA adapter and a pointer head on `Qwen/Qwen3.5-4B-Base`. A request contains a state (the entity and context) and typed questions; for a multiple-choice question the model runs a single prefill pass and returns a probability for every option, using a pointer head over option tokens. Questions are isolated from each other, so a SNOMED question cannot influence an ICD question. Each checkpoint ships a fitted temperature for calibrated probabilities; the pointer head applies it at load time.

5.2 Input construction

The state is a JSON object with `entity` and `context` (context truncated to 400 characters at export). Each question has instructions (*"You are a certified medical coder. Choose the ICD-10-CM code that best matches the clinical concept in the patient context."*), a list of candidate codes, and an 80-character terminology description per candidate. The shortlist size K is small by design: Kev-4B was validated for state+question lengths of about 1,024 tokens, and larger candidate sets add near-gold decoys (see ablations).

5.3 Training data

Training rows are built with the exact inference recipe: the same dense pool, fusion and judge blend produce the shortlist; rows whose gold is not in the shortlist are dropped. This yields 1,776 training pairs (rows × systems). Option order is shuffled per example. The label is the option name. No generated rationales are used.

5.4 Training protocol

Backbone	`Qwen/Qwen3.5-4B-Base` (frozen)
Initialisation	`jaredpalmer/kev-4b` (LoRA + head)
Adapter	LoRA, rank 16, $\alpha = 32$; pointer head trained
Epochs	2
Learning rate	2×10^{-5}
Batch / accumulation	1 / 8
Precision	bf16 (gradient checkpointing on)
Hardware	one RTX 6000 Ada (48 GB); 44 minutes for the canonical run

LoRA training cannot be resumed mid-run in the version used; on the shared host we therefore also built a segmented variant (seven one-epoch shards, warm-starting each segment from the previous checkpoint) which reproduces the full-run accuracy within one point and survives periodic SIGKILLs.

6 Stage 2c: permutation ensembling

Listwise decision models are order-sensitive: changing the option order can change the answer. The Kev server exposes a `/v1/systemone/permute` endpoint that reruns one choice question under n option orders. Let $\pi_1, \ldots, \pi_n$ be the orders (the first is the presented order) and $p_t(c)$ the calibrated probability of candidate c under order t. The ensemble distribution is

$$\bar{p}(c) = \frac{1}{n} \sum_{t=1}^{n} p_t(c),$$

the prediction is $\arg\max_c \bar{p}(c)$, and we record

$$\text{margin} = \bar{p}_{(1)} - \bar{p}_{(2)}, \qquad \text{stable} = \mathbb{1}\big[\arg\max_c p_t(c) \text{ is the same for all } t\big].$$

Both are used for abstention/routing. The cost is linear in n; n=8 was chosen because n=5 already captured most of the gain (dev ICD 0.565, SNOMED 0.667) and beyond 8 the improvement was inside noise.

7 Decision layer and abstention

The system always emits its best code; in addition it reports the margin and stability. Ordering rows by margin gives a monotone coverage–accuracy curve (Table 4), which supports two operating modes: *auto-map* the confident rows and *review* the rest. A deployment can fix a review budget and read the expected accuracy from this curve. We do not claim that the calibrated probability is an error probability for every row — convention failures can be confident — so stability is used alongside the margin.

8 Experimental protocol

All choices were made on the dev split (432 rows, 780 questions). The test split (577 rows, 1,025 questions) was held out. Config B (ICD K=20 with the combined judge) was selected after observing that the ICD shortlist recall on test dropped to 71.2% under config A; its dev accuracy is 0.537/0.633, i.e. the configuration is slightly worse on dev and better on test. Both configurations and both splits are reported; no test rows were used to train or tune any model parameter.

9 Results

9.1 Headline numbers

Split	Configuration	ICD	SNOMED	Overall
dev	A, single pass	0.548	0.647	0.598
dev	A, permute n=5	0.565	0.667	0.616
dev	**A, permute** n=8	**0.573**	**0.667**	**0.620**
test	A, single pass	0.532	0.603	0.571
test	B, single pass	0.551	0.592	0.571
test	**B, permute** n=8	**0.564**	**0.612**	**0.590**

Table 2: Final results.

9.2 Where the accuracy goes

	dev (A)		test (B)	
	ICD	SNOMED	ICD	SNOMED
Shortlist recall	0.818	0.887	0.838	0.874
acc|present	0.700	0.751	0.673	0.700
Exact match	0.573	0.667	0.564	0.612
3-char block (ICD)	0.741	–	0.748	–
Unseen gold (count)	115	265	146	319
accuracy	0.444	0.623	0.507	0.564

Table 3: Accuracy decomposition and long-tail diagnostics.

Reading the decomposition. ICD loses more to retrieval than SNOMED (recall 0.818 vs. 0.887 on dev). The chooser is stronger on SNOMED than on ICD even though SNOMED has more unseen golds: the SNOMED label space is more semantically distinct, while ICD's near-duplicate leaf codes create convention traps. Partial credit confirms this: 0.748 of test ICD answers are in the correct three-character block but only 0.564 are exact.

9.3 Coverage/abstention

Coverage	100%	90%	80%	70%	60%	50%
dev ICD	0.573	0.607	0.648	0.693	0.747	0.796
dev SNOMED	0.667	0.704	0.736	0.766	0.812	0.856
test ICD	0.564	0.594	0.634	0.679	0.721	0.761
test SNOMED	0.612	0.649	0.690	0.740	0.770	0.809

Table 4: Accuracy at a given coverage when ranking by the ensemble margin (thresholded after ensembling).

10 Ablations

Variant	ICD	SNOMED
Final (A + permute n=8)	**0.573**	**0.667**
No permutation ensemble (single pass)	0.548	0.647
Permutation n=5	0.565	0.667
ICD K=15 (UltraBERT judge), dev	0.543	–
ICD K=20 (combined judge), dev	0.537	0.633
ICD K=30 (combined judge), dev	0.530	0.594
SNOMED K=40 (combined judge)	–	0.633
Combined judge for both systems (K=10/30)	0.554	0.638
Model ensemble (canonical + earlier chooser, n=4+4)	0.581	0.650
Criteria length 120 chars	0.545	0.647
Full context (1,500 chars)	0.545	0.647

Table 5: Ablations on dev. The model ensemble helps ICD but hurts SNOMED; the two models are strongly correlated.

Why not a larger K? Recall increases with K, but every additional candidate is, by construction, a judge-preferred near-gold decoy. On dev the chooser's conditional accuracy falls faster than recall rises. On test, where recall was 10 points lower, the trade-off reverses — which is why config B widens only the ICD shortlist. This is a distribution effect, not a modelling one, and it is the clearest evidence for monitoring shortlist recall in deployment.

Why not a larger chooser? A 9B Kev fine-tune was prepared (same data and recipe, `kev-9b` initialisation); the shared-GPU environment repeatedly killed the run and LoRA training has no resume point, so the 4B remains the submitted model. The serving API and training pipeline are unchanged for 9B/27B, so this is an operational rather than architectural limitation. The submitted 4B checkpoint is public at https://huggingface.co/d3let3/kev-4b-clinical-mapping.

11 Error analysis

11.1 By stage

On dev with the canonical chooser: ICD has 66 retrieval failures and 89 chooser failures; SNOMED 47 and 92. Chooser failures dominate. 61 of the 155 ICD errors (39%) are same-three-character-block siblings; in a manual sample of 49 such errors from the earlier configuration, only 6 had a distinguishing cue in the context. This is the signature of labelling convention and under-specified notes rather than of insufficient model capacity: the model cannot infer a detail the note does not state, and it disagrees with the annotator when the annotator follows a convention.

11.2 Qualitative cases

Entity	Failure mode	Behaviour	Gold / pick
hypercholesteremia	parent vs. billable child	confident, stable	E78.00 / E78.0
right lower lobe pulmonary nodule	sibling specificity	low margin, unstable	R91.1 / R91.8
lee	abbreviation collision	low margin (0.013)	I89.0 / A57
t2dm poorly controlled	specificity; order sensitivity	single pass wrong, ensemble correct	E11.65 / E11.65
poorly differentiated carcinoma	unspecified vs. specified neoplasm	single pass C80, ensemble C80.1	C80.1 / C80.1
ø tremor	OCR artifact	correct ICD, SNOMED slip	41647002 / 26079004

Table 6: Hard cases from the demo. Confidence is informative but not a guarantee: the first row is a confident convention failure.

11.3 Long tail

Accuracy on gold codes never seen in training is 0.444 (dev ICD), 0.507 (test ICD), 0.623 (dev SNOMED), 0.564 (test SNOMED). For SNOMED, unseen codes are the majority of questions and the system still exceeds 0.55, which indicates that the judge and chooser generalise from descriptions rather than memorising codes. The ICD unseen bucket is the weakest: retrieval failures concentrate there, because rare codes have fewer, shorter descriptions and weaker lexical overlap.

12 Deployment characteristics

- **Footprint.** Chooser 4B in bf16 (≈10 GB) on one 24 GB-class GPU; the judge stage needs a 150M encoder and optionally the 8B reranker. The retrieval engine runs on CPU (FAISS flat indexes) and can be pre-computed per batch.
- **Latency.** Single pass: 14–16 questions/s on one RTX 6000 Ada in our evaluation loop, i.e. roughly 65–75 ms per question including HTTP. Permutation ensembling with n=8 costs eight prefills: about 1.7 s per question end-to-end in our client loop, which is well below chart-processing budgets when batched per document.
- **Determinism.** The chooser has no sampling: probabilities come from a single prefill, and permutation orders are seeded. Runs are reproducible; server restarts do not change the model.

- **Batching.** Questions are isolated rows that share the state prefill, so a document with many entities amortises the state computation across questions.
- **Abstention.** Margin and stability thresholds are configuration, not model changes; the coverage curve above quantifies the trade-off for a review budget.

13 Limitations and future work

1. **Conventions.** The largest error group is dataset-specific specificity convention. A parent/child tie-break learned from training labels is the most promising next step; it is a post-layer, not a new model.
2. **ICD retrieval on test.** Recall 0.838 at K=20 caps ICD accuracy more than anything else. A recall-oriented ICD judge plus sibling expansion of the top candidate is the direct fix.
3. **Evidence-absent cases.** For notes that do not state the detail separating two leaves, a calibrated abstention (route to review) is more honest than a guess; the coverage curves show the value.
4. **Larger chooser.** 9B/27B are drop-in; the only blocker was the shared GPU environment.
5. **SNOMED partial credit.** We report exact match only; an IS-A distance metric would separate wrong-concept from wrong-specificity errors, which the partial-credit analysis already suggests is a meaningful share.

A Hyperparameters

Candidate caps	exact 50, fuzzy 50, lexical 200, dense 200, memory 100, crosswalk 50, char-ngram 200
Dense models	gte-base, BioLORD-2023, UltraBERT-Embed-150M; UltraBERT source weight 0.84
Judge	UltraBERT-150M pairwise, 60.6k pairs, 20 hard negatives, 2 epochs @ 4×10^{-5}
Judge blend	$0.5\, z_u + 0.5\, z_q$ per system
Shortlist (A)	ICD w=0.2, K=10; SNOMED w=0.6, K=30
Shortlist (B)	ICD w=0.6 (combined), K=20; SNOMED unchanged
Chooser	Kev-4B, LoRA r=16, α=32, 2 epochs, lr 2×10^{-5}, batch 1×8
State	entity + context (400 chars); option description 80 chars
Ensemble	n=8 seeded option orders; mean calibrated probability
Coverage policy	route by margin/stability; see Table 4

B Reproduction commands

```
# single command: reproduce submitted predictions
python predict.py --input data/test_input.csv --output predictions/test_predictions.csv

# live two-stage run with permutation ensembling
KEV_ENDPOINT=http://127.0.0.1:8013 python predict.py \
    --input data/test_input.csv --output /tmp/live.csv --mode kev --nperm 8

# judge scores (resumable), shortlist, and evaluation
python scripts/16_rerank_scores.py --model Qwen/Qwen3-Reranker-8B \
    --jsonl data/dev_shortlist.jsonl --meta data/dev_shortlist.meta.jsonl \
    --out /tmp/qwen_scores.jsonl
python scripts/09_eval_kev.py --jsonl data/dev_shortlist.jsonl \
    --url http://127.0.0.1:8013 --out /tmp/dev_single.csv
python scripts/09b_eval_permute.py --jsonl data/dev_shortlist.jsonl \
    --csv /tmp/dev_single.csv --url http://127.0.0.1:8013 --nperm 8 \
    --out /tmp/dev_permute8.csv
```